CS543 Web Services

Assignment 3 — Integrate an External SOAP Partner

Team Information

Team ID	8
Project	CampusEats — Campus Food Ordering System
External Partner	UniPay Payment Gateway (design-time / fictional)
Operation	charge

#	Name	Roll No.	Role
1.	Priya	`20251651073`	Team Leader
2.	Nandan Kabra	`20251651060`	Member
3.	Abhinav Gupta	`20251651002`	Member
4.	Neeraj Sharma	`20251651063`	Member

Scope note. This submission is a design-time artefact set. UniPay is a fictional external partner created for this assignment; `api.unipay.example.com` is a reserved-domain placeholder. No SOAP server was built or deployed, no UDDI registry was stood up, no live payment provider was contacted, and no real payment or API calls were made. No unrelated CampusEats application code was modified.

Deliverables in this submission

File	Contents
integration.pdf	This document — Context, HTTP Binding, Discovery, Fault Mapping
partner.wsdl	Complete SOAP 1.1 / HTTP WSDL for the UniPay charge operation
soap-request.xml	SOAP request envelope (credentials header + charge body)
soap-response.xml	Successful SOAP response (transactionId, status)
soap-fault.xml	SOAP Fault for a declined card (card_declined)

1. Context

CampusEats is a campus food-ordering system decomposed into five internal microservices — Identity, Catalogue, Orders, Payments and Delivery — which communicate with each other and with student-facing clients over REST/JSON. The Payments service is the only component that reaches outside the campus boundary: to actually move money it must call an external card acquirer. For this assignment that acquirer is **UniPay Payment Gateway**, an external partner that exposes a single `charge` operation over SOAP 1.1 carried on HTTP.

The charge operation at a glance

Partner / Business	`UniPay`
Service	`UniPay Payment Gateway`
Operation	`charge`
Protocol	`SOAP 1.1 over HTTP, document/literal`
SOAP header	`merchantId, apiKey`
Request body	`orderId, amount, currency, cardToken`
Success response	`transactionId, status`
Declared fault	`ChargeFault (e.g. card_declined)`
Target namespace	`http://unipay.example.com/payment/v1`

Why SOAP is appropriate at this external payment edge

This particular edge is a cross-organisational, money-moving, transaction-oriented integration between two independently operated systems, which is exactly the setting SOAP and WS-* were designed for. The WSDL gives both parties a single machine-readable contract that pins down operation names, field types and cardinality, so the request shape can be validated and client stubs generated before a single message is exchanged — valuable when the partner cannot be asked to explain undocumented behaviour. SOAP's separate header block lets merchant identity (`merchantId`, `apiKey`) travel as structured, independently processable metadata rather than being mixed into the business payload, and that same envelope structure is what WS-Security extends for message-level signing and encryption — protection that survives intermediaries and TLS termination rather than ending at the transport hop. Finally, SOAP faults give payment failures a standardised, typed error channel declared in the contract itself, so a declined card arrives as a fault the adapter must handle rather than as an ad-hoc error field the caller might silently ignore.

Why the rest of CampusEats stays REST

Only the outward-facing payment edge speaks SOAP. CampusEats is **not** being converted from REST to SOAP. Internal traffic — browsing the catalogue, placing an order, tracking a delivery, authenticating a student — is resource-oriented, high-volume, latency-sensitive and consumed by browser and mobile clients, for which REST/JSON remains lighter, easier to cache and far simpler to consume. The protocol difference is deliberately contained inside a single **SOAP integration adapter** owned by the Payments service. That adapter is the only code in the system that constructs a SOAP envelope, and it translates in both directions: the internal REST payment request becomes a SOAP `charge` call, and the SOAP response or fault becomes a CampusEats-defined result. Protocol choice is therefore an integration detail at one boundary, not an architectural style imposed on the whole system.

2. HTTP Binding

The WSDL binds the `charge` operation to SOAP 1.1 with transport `http://schemas.xmlsoap.org/soap/http`, so every invocation is a single HTTP **POST** whose entity body is the SOAP envelope. SOAP 1.1 requires the `SOAPAction` HTTP header to be present and its value to be a quoted string; the quotes are HTTP-level framing, so the WSDL attribute `soapAction="charge"` is carried on the wire as `SOAPAction: "charge"`.

Field	Value
HTTP method	`POST`
Request path	`/soap/payment`
HTTP version	`HTTP/1.1`
Host	`api.unipay.example.com`
Content-Type	`text/xml; charset=utf-8`
SOAPAction	`"charge"`
Endpoint	`https://api.unipay.example.com/soap/payment`
Message body	`SOAP envelope — see soap-request.xml`

Request — HTTP POST carrying the SOAP envelope

```
POST /soap/payment HTTP/1.1
Host: api.unipay.example.com
Content-Type: text/xml; charset=utf-8
SOAPAction: "charge"
Content-Length: 512
Accept: text/xml

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
    xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:pay="http://unipay.example.com/payment/v1">
  <soapenv:Header>
    <pay:Credentials>
      <pay:merchantId>CAMPUSEATS001</pay:merchantId>
      <pay:apiKey>sk_test_unipay_9f3c1a7b2d</pay:apiKey>
    </pay:Credentials>
  </soapenv:Header>
  <soapenv:Body>
    <pay:ChargeRequest>
      <pay:orderId>CE1001</pay:orderId>
      <pay:amount>850.00</pay:amount>
      <pay:currency>INR</pay:currency>
      <pay:cardToken>tok_test_12345</pay:cardToken>
    </pay:ChargeRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

The body above is exactly the accompanying `soap-request.xml` deliverable. Note that the card is represented only by a token (`tok_test_12345`) — raw card numbers never enter CampusEats.

Success response — HTTP 200

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
    xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:pay="http://unipay.example.com/payment/v1">
  <soapenv:Body>
    <pay:ChargeResponse>
      <pay:transactionId>UP20260819001</pay:transactionId>
      <pay:status>APPROVED</pay:status>
    </pay:ChargeResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

Fault response — HTTP 500

SOAP 1.1 requires a fault to be returned with HTTP status 500. The declined-card case therefore arrives as a well-formed SOAP Fault, not as a 200 response carrying an error flag:

```
HTTP/1.1 500 Internal Server Error
Content-Type: text/xml; charset=utf-8

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
    xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:pay="http://unipay.example.com/payment/v1">
  <soapenv:Body>
    <soapenv:Fault>
      <faultcode>soapenv:Client</faultcode>
      <faultstring>Payment declined by issuing bank</faultstring>
      <faultactor>https://api.unipay.example.com/soap/payment</faultactor>
      <detail>
        <pay:ChargeFault>
          <pay:faultCode>card_declined</pay:faultCode>
          <pay:faultReason>The issuing bank declined this card.</pay:faultReason>
          <pay:orderId>CE1001</pay:orderId>
        </pay:ChargeFault>
      </detail>
    </soapenv:Fault>
  </soapenv:Body>
</soapenv:Envelope>
```

3. Discovery

CampusEats needs a repeatable way to answer three questions about UniPay: who the business is, where its service lives, and where the technical contract that describes it can be fetched. Classical SOAP tooling answered this with a public UDDI registry, but public UDDI was retired in 2006 and modern practice does not depend on it. **No UDDI server is built, deployed or required for this assignment.**

Discovery model used

CampusEats uses a **service catalogue / registry**: a maintained inventory of the external services the system depends on. Each catalogue record names the partner business, the service, its endpoint, and a pointer to the technical contract. There are consequently two equivalent ways for a CampusEats developer or build pipeline to obtain the WSDL:

- **Direct WSDL URL.** UniPay publishes the contract at a well-known address, conventionally the endpoint with a `?wsdl` query string. If that URL is already known it can be fetched straight away and used to generate client stubs.
- **Through the catalogue.** If only the partner or capability is known — "who provides card payments?" — the catalogue is searched by business or service name, and the record yields the endpoint and the WSDL pointer. This is the UDDI *idea* (business → service → binding → technical model) preserved without the UDDI *infrastructure*.

The catalogue record is the versioned, reviewable source of truth. The endpoint is never hard-coded in CampusEats source; it is configuration sourced from this record, so a partner endpoint change is a catalogue and configuration update rather than a code change.

Catalogue / registry record — UniPay

Field	Value
Business	`UniPay`
Service	`UniPay Payment Gateway`
Endpoint	`https://api.unipay.example.com/soap/payment`
WSDL / tModel-style pointer	`https://api.unipay.example.com/soap/payment?wsdl`

Read against the classic registry model, the four fields map cleanly onto the concepts a UDDI inquiry would have returned:

Field	What it lets CampusEats discover
Business	The organisation CampusEats has a commercial relationship with — the equivalent of a UDDI businessEntity. Answers *who* provides the capability.
Service	The named capability that business offers — the equivalent of a businessService. Answers *what* is provided.
Endpoint	The concrete network address where the service is invoked — the equivalent of a bindingTemplate access point. Answers *where* to send the POST.
WSDL pointer	The technical contract describing operations, message shapes and faults — the equivalent of a tModel reference. Answers *how* to call it correctly.

Together these let CampusEats move from "we need card payments" to a generated, contract-checked client without any out-of-band documentation: the record identifies the business and service, the endpoint tells the adapter where to send the request, and the WSDL pointer supplies the machine-readable contract. Because the WSDL is retrieved at design time and the generated stubs are committed, a partner contract change is caught during the build rather than at runtime. In this assignment the WSDL is supplied directly as the `partner.wsdl` deliverable; the `?wsdl` URL is a design-time placeholder and is not live.

4. Fault Mapping

When UniPay declines a card it returns a SOAP Fault whose detail carries the partner's own error code, `card_declined`. That string belongs to UniPay's vocabulary, not to CampusEats. The SOAP integration adapter inside the Payments service is responsible for translating it into the error defined by the CampusEats contract, `PAYMENT_DECLINED`, before anything reaches a CampusEats consumer.

Mapping

UniPay partner fault code	CampusEats internal error code
card_declined	`PAYMENT_DECLINED`

Translation flow

UniPay (external partner)

```
SOAP Fault — faultcode soapenv:Client, detail/ChargeFault/faultCode = card_declined
```

↓

CampusEats SOAP integration adapter

```
Parses the fault detail and maps the partner code to the internal code PAYMENT_DECLINED
```

↓

CampusEats API (student-facing contract)

```
HTTP 402  { "error": "PAYMENT_DECLINED", "message": "Payment could not be completed."
}
```

The student sees only the CampusEats-defined message, **"Payment could not be completed."** The internal code `PAYMENT_DECLINED` is what the CampusEats API contract publishes to client applications; the human-readable sentence is what the UI renders.

Why the partner vocabulary must stay behind the integration boundary

- **Contract stability.** UniPay owns `card_declined` and can rename, split or retire it at any time. If that string were exposed through the CampusEats API, a partner-side change would silently become a breaking change for every CampusEats client.
- **Partner replaceability.** Swapping UniPay for a different acquirer must be an adapter-level change. If clients branch on partner codes, the partner is effectively welded into the public contract and cannot be replaced without breaking consumers.
- **Leakage of internal architecture.** Partner-specific codes tell an outside observer which gateway CampusEats uses and how the integration is wired — information that has no value to a student and some value to an attacker.
- **Information disclosure.** Raw gateway diagnostics can hint at why an issuer declined a card. Collapsing distinct partner codes onto one neutral consumer-facing error avoids exposing detail about a cardholder's account that CampusEats has no business relaying.
- **Consistent client handling.** A closed, documented set of CampusEats error codes lets clients implement one predictable retry-or-inform path, instead of maintaining an ever-growing table of third-party strings.

The same discipline applies to every other partner code. Additional UniPay faults declared in the WSDL (`insufficient_funds`, `invalid_card_token`, `merchant_not_authorised`, `duplicate_order`) are each mapped to a CampusEats code by the adapter, and any unrecognised or unmapped partner fault is mapped to a safe default such as `PAYMENT_FAILED` rather than being passed through verbatim. The full partner fault — including the original code and UniPay's reason text — is written to the Payments service logs with a correlation id, so operators retain the diagnostic detail that students never see.